

Contents

- Introduction of Binary Codes
- Representation of Numbers in different bases
- Necessity of Binary Codes
 - Binary-Coded Decimal Code (BCD)
 - Excess-3 Code
 - Gray Code
 - Error Detection Codes
 - Non-numeric Binary Codes
 - Alphanumeric Codes
 - ASCII Codes

Introduction of Binary Codes

- ❖ **Binary Codes:** Any discrete element of information that is distinct among a group of quantities can be represented with a binary code (i.e., a pattern of 0's and 1's).
- The codes must be in binary because, in today's technology, only circuits that represent and manipulate patterns of 0's and 1's can be manufactured economically for use in computers.
- However, it must be realized that binary codes merely change the symbols, not the meaning of the elements of information that they represent.
- For example, the binary number **1000001** represents **65** (Decimal) in straight binary, **41** in (BCD) and Alphabet **A** in **ASCII** code.

Necessity of Binary Codes

❖ Necessity of Binary Codes:

- There is a wide variety of binary codes used in digital systems.
- Often it is required to convert from one code to another.
- For example the input to a digital system may be in natural BCD and output may be 7-segment LEDs. The digital system used may be capable of processing the data in straight binary format.
- Therefore, the data has to be converted from one type of code to another type for different purpose.
- Big string of data can easily converted to 0 or 1.

Binary-Coded Decimal Code (BCD)

1. Binary-Coded Decimal Code (BCD)

- Each Decimal Digit is represented by 4 bits.
- A number with k decimal digits will require $4k$ bits in BCD. Decimal 396 is represented in BCD with 12 bits as 0011 1001 0110, with each group of 4 bits representing one decimal digit.
- A decimal number in BCD is the same as its equivalent binary number only when the number is between 0 and 9.
- A BCD number greater than 10 looks different from its equivalent binary number, even though both contain 1's and 0's.
- Example of BCD:

$$(185)_{10} = (0001\ 1000\ 0101)_{\text{BCD}} = (10111001)_2$$

Binary-Coded Decimal Code	
Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Binary-Coded Decimal Code (BCD)

❖ Binary-Coded Decimal Code (BCD)

- The BCD code is the 8,4,2,1 code.
- 8, 4, 2, and 1 are weights
- BCD is a *weighted code*
- This code is the simplest, most intuitive binary code for decimal digits and uses the same powers of 2 as a binary number, but only encodes the first ten values from 0 to 9.
- Example: $1001 (9) = 1000 (8) + 0001 (1)$
- How many “invalid” code words are there?
- What are the “invalid” code words?

Binary Power	2^3	2^2	2^1	2^0
Binary Weight:	8	4	2	1

Warning: Conversion or Coding?

- ❖ Do **NOT** mix up conversion of a decimal number to a **binary number** with coding a decimal number with a **BINARY CODE**.
- $13_{10} = 1101_2$ (This is conversion)
- $13 \Leftrightarrow 0001|0011$ (This is coding)

Binary-Coded Decimal Code (BCD)

❖ Binary-Coded Decimal Code Addition

BCD: One decimal digit + one decimal digit.

- If the result is 1 decimal digit (≤ 9), then it is a simple binary addition.

$$\begin{array}{r} 5 \quad \quad 0101 \\ + 3 \quad + 0011 \\ \hline \end{array}$$

8 $\longleftrightarrow$ 1000

- If the result is two decimal digits (≥ 10), then binary addition gives invalid combinations.

$$\begin{array}{r} 5 \quad \quad 0101 \\ + 5 \quad + 0101 \\ \hline \end{array}$$

0001 0000 $\longleftrightarrow$ 10 1010

Binary-Coded Decimal Code (BCD)

❖ Binary-Coded Decimal Code Addition

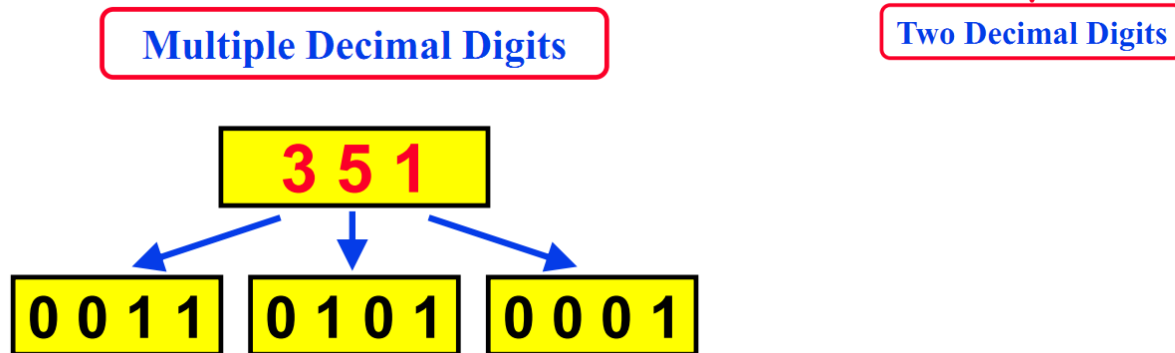
- If the binary result is greater than 9, correct the result by adding 6

$$\begin{array}{r} 5 \\ + 5 \\ \hline 10 \end{array}$$

10

$$\begin{array}{r} 0101 \\ + 0101 \\ \hline 1010 \\ + 0110 \\ \hline 0001 \quad 0000 \end{array}$$

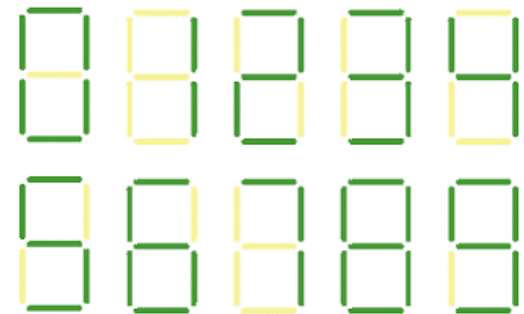
For multiple digit



Binary-Coded Decimal Code (BCD)

❖ Applications of BCD:

- Some early computers processed BCD numbers. Arithmetic operations can be performed using this code. Input to a digital system may be in natural BCD and output may be 7-segment LEDs.
- It is observed that more number of bits are required to code a decimal number using BCD code than using the straight binary code. However in spite of this disadvantage it is very convenient and useful code for input and output operations in digital systems.



Excess-3 Code

2. Excess-3 Code:

This is another form of BCD code, in which each decimal digit is coded into a 4-bit binary code.

- The code for each decimal digit is obtained by adding decimal 3 to the natural BCD code of the digit.
- For example, decimal 2 is coded as $0010 + 0011 = 0101$ in Excess-3 code.
- It is not a weighted code.
- It is a self-complementing code, which means 1's complement of the coded number yields 9's complement of the number itself.
- Self complementing property of this helps considerably in performing subtraction operation in digital systems, so this code is used for certain arithmetic operations.

Excess-3 Code

❖ Excess-3 Code:

- **BCD To Excess – 3 Code Conversions:**

Convert BCD 2 i. e. 0010 to Excess – 3 codes

For converting 4 bit BCD code to Excess – 3, add 0011 i. e. decimal 3 to the respective code using rules of binary addition.

$$0010 + 0011 = 0101 - \text{Excess} - 3 \text{ code for BCD } 2$$

- **Excess – 3 Code To BCD Conversion:**

The 4 bit Excess-3 coded digit can be converted into BCD code by subtracting decimal value 3 i.e. 0011 from 4 bit Excess-3 digit.

e.g. Convert 4-bit Excess-3 value 0101 to equivalent BCD code.

$$0101 - 0011 = 0010 - \text{BCD for } 2$$

Gray Code

3. Gray Code:

- It is a modified binary code in which a decimal number is represented in binary form in such a way that each Gray- Code number differs from the preceding and the succeeding number by a single bit.
- For example, for decimal number 5 the equivalent Gray code is 0111 and for 6 it is 0101. These two codes differ by only one bit position i. e. third from the left.)
- Whereas by using binary code there is a possibility of change of all bits if we move from one number to other in sequence (e.g. binary code for 7 is 0111 and for 8 it is 1000).
- Therefore it is more useful to use Gray code in some applications than binary code.
- The Gray code is a non-weighted code
- It is also known as reflected code.

Gray Code

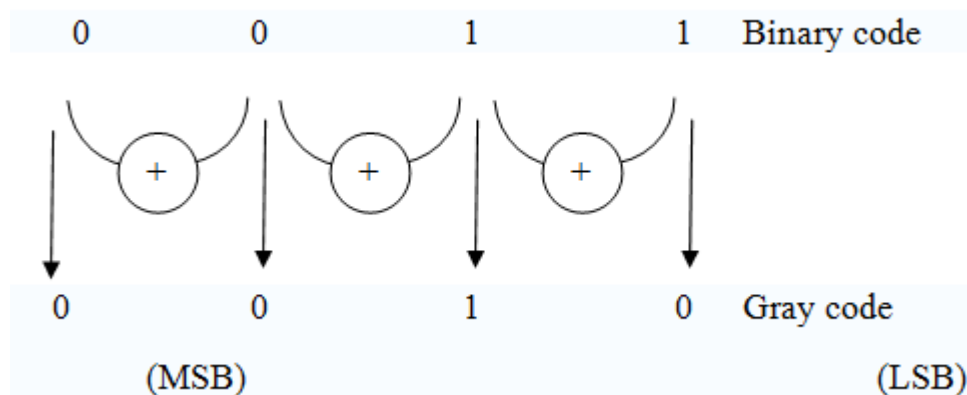
❖ Applications of Gray Code:

1. Important feature of Gray code is it exhibits only a single bit change from one code word to the next in sequence. This property is important in many applications such as Shaft encoders where error susceptibility increases with number of bit changes between adjacent numbers in sequence.
2. It is sometimes convenient to use the Gray code to represent the digital data converted from the analog data (Outputs of ADC).
3. Gray codes are used in angle-measuring devices in preference to straight forward binary encoding.
4. Gray codes are widely used in K-map
5. The disadvantage of Gray code is that it is not good for arithmetic operation

Gray Code

❖ Binary to Gray Conversion

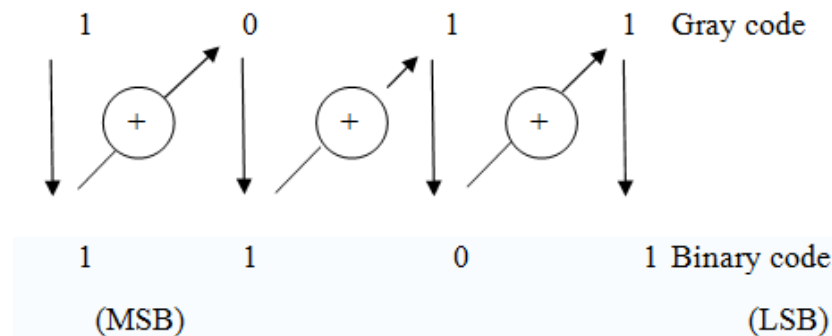
- In this conversion, the input straight binary number can easily be converted to its Gray code equivalent.
 1. Record the most significant bit as it is.
 2. EX-OR this bit to the next position bit, record the resultant bit.
 3. Record successive EX-ORed bits until completed.
 4. Convert 0011 binary to Gray.



Gray Code

❖ Gray to Binary Conversion

1. The Gray code can be converted to binary by a reverse process.
2. Record the most significant bit as it is.
3. EX-OR binary MSB to the next bit of Gray code and record the resultant bit.
4. Continue the process until the LSB is recorded.
5. Convert 1011 Gray to Binary code.



Different Binary Codes Format

- ❖ There are over 8,000 ways that you can chose 10 elements from the 16 binary numbers of 4 bits. A few are useful:

Decimal	8,4,2,1	Excess3	8,4,-2,-1	Gray
0	0000	0011	0000	0000
1	0001	0100	0111	0100
2	0010	0101	0110	0101
3	0011	0110	0101	0111
4	0100	0111	0100	0110
5	0101	1000	1011	0010
6	0110	1001	1010	0011
7	0111	1010	1001	0001
8	1000	1011	1000	1001
9	1001	1100	1111	1000

Error Detection Codes

4. Error Detection Codes

- Redundancy (e.g. extra information), in the form of extra bits, can be incorporated into binary code words to detect and correct errors.
- A simple form of redundancy is parity, an extra bit appended onto the code word to make the number of 1's odd or even. Parity can detect all single-bit errors and some multiple-bit errors.
- **Parity Bit:** A parity bit is an extra bit included with a message to make the total number of 1's either even or odd.
- A code word has even parity if the number of 1's in the code word is even.
- A code word has odd parity if the number of 1's in the code word is odd.

	With even parity	With odd parity
ASCII A = 1000001	01000001	11000001
ASCII T = 1010100	11010100	01010100

Error Detection Codes

❖ Error Detection Codes

- In each case, we insert an extra bit in the leftmost position of the code to produce an even number of 1's in the character for even parity or an odd number of 1's in the character for odd parity. In general, one or the other parity is adopted, with even parity being more common.

❑ **Even Parity:** For ASCII characters, bit 7 (that is, the eight-bit, the most significant bit) is set so that the total number of 1s in a character is even, as in:

'A' = 100 0001 Parity = 0; Code = 0100 0001; Parity Bit = 0

'C' = 100 0011 Parity = 1; Code = 1100 0011; Parity Bit = 1

❑ **Odd Parity:** The parity bit is set so that the total number of 1s is odd, as in:

'A' = 100 0001 Parity = 1 Code = 1100 0001; Parity bit = 1

If we consider for example 'A' with even parity 0100 0001, any 1-bit error will change the number of 1s and 0s in the symbol, and thus, the parity of the symbol.

Non-numeric Binary Codes

- ❖ Given n binary digits (called bits), a binary code is a mapping from a set of represented elements to a subset of the 2^n binary numbers
- ❖ Example: A binary code for the seven colors of the rainbow.

Color	Binary Number
Red	000
Orange	001
Yellow	010
Green	011
Blue	101
Indigo	110
Violet	111

- ❖ Code 100 is not used

Introduction of Binary Codes

5. Alphanumeric Codes

- Represent numbers and alphabetic characters.
- Characters need to be represented. In particular, they need to be represented in binary. After all, computers store and manipulate 0's and 1's (and even those 0's and 1's are just abstractions. The implementation is typically voltages).
- The ASCII is the most common alphanumeric code.

6. ASCII: American Standard Code for Information Interchange

- ASCII has 128 characters and symbols represented by a 7-bit binary code.
- It can be considered an 8-bit code with the MSB always 0.
 - 94 Graphic printing characters.
 - 34 Non-printing characters
- Some non-printing characters are used for text format (e.g. BS = Backspace, CR = carriage return)

Introduction of Binary Codes

❖ ASCII Properties

ASCII has some interesting properties:

- Digits 0 to 9 span Hexadecimal values 30_{16} to 39_{16}
- Upper case A-Z span 41_{16} to $5A_{16}$
- Lower case a-z span 61_{16} to $7A_{16}$
- Lower to upper case translation (and vice versa) occurs by occurs by

Introduction of Binary Codes

❖ ASCII Code

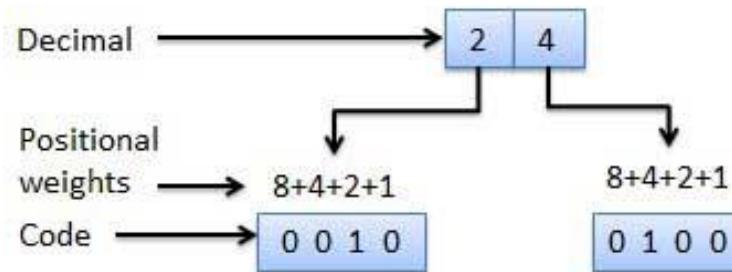
$B_4 B_3 B_2 B_1$	$B_7 B_6 B_5$							
	000	001	010	011	100	101	110	111
0000	NULL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Tutorial Solve

Problem 01: What are weighted codes and non-weighted codes?

Solution:

- In weighted codes each digit position of number represents a specific weight. The codes 8421, 2421, and 5211 are weighted codes.



- Non weighted codes are not assigned with any weight to each digit position i.e. each digit position within the number is not assigned a fixed value. Gray code, Excess-3 code are non-weighted code.

Tutorial Solve

Problem 02: Why is Excess-3 code called as self-complementing code?

Solution:

- A self complementing code is one that's 9's complement in decimal is the 1's complement in binary.
- Excess-3 code is called self-complementing code because 9's complement of a coded number can be obtained by just complementing each bit.

Problem 03: What is invalid BCD?

Solution:

With four bits, sixteen numbers (0000 to 1111) can be represented, but in BCD code only 10 of these are used as decimal numbers have only 10 digits from 0 to 9. The six code combinations (1010 to 1111) are not used and are invalid.

Tutorial Solve

Problem 05: Represent the decimal number 5,137 in (a) BCD, (b) excess-3 code, (c) 2421 code, and (d) a 6311 code.

Solution:

- (a) 5,137 BCD: 0101_0001_0011_0111 10
- (b) Excess-3: 1000_0100_0110_1010
- (c) 2421: 1011_0001_0011_0111
- (d) 6311: 0111_0001_0100_1001

Tutorial Solve

Problem 06: Find the 9's complement of decimal 6,248 and express it in 2421 code. Show that the result is the 1's complement of $(6,248)_{2421}$. This demonstrates that the 2421 code is self-complementing.

Solution:

- Here, 9's complement of decimal 6,248 = 3, 751.
- 2421 code of 3, 751 = **0011-1101-1011- 0001**
- 2421 code of 6,248 = 1100 - 0010 - 0100-1110
- 1's complement of 1100 - 0010 - 0100-1110 = **0011-1101-1011- 0001**